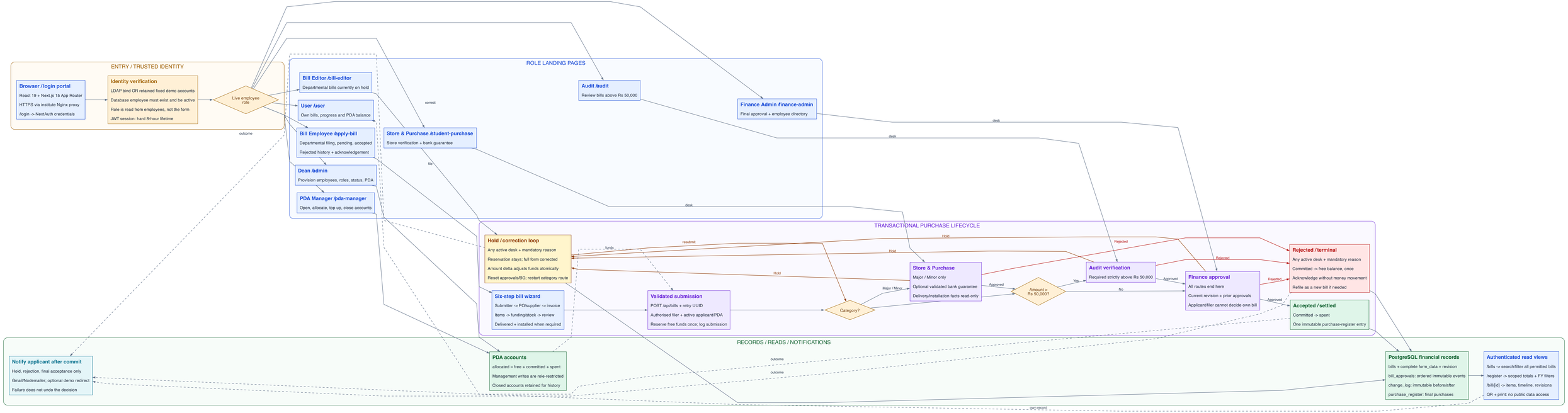
01 Complete website flow

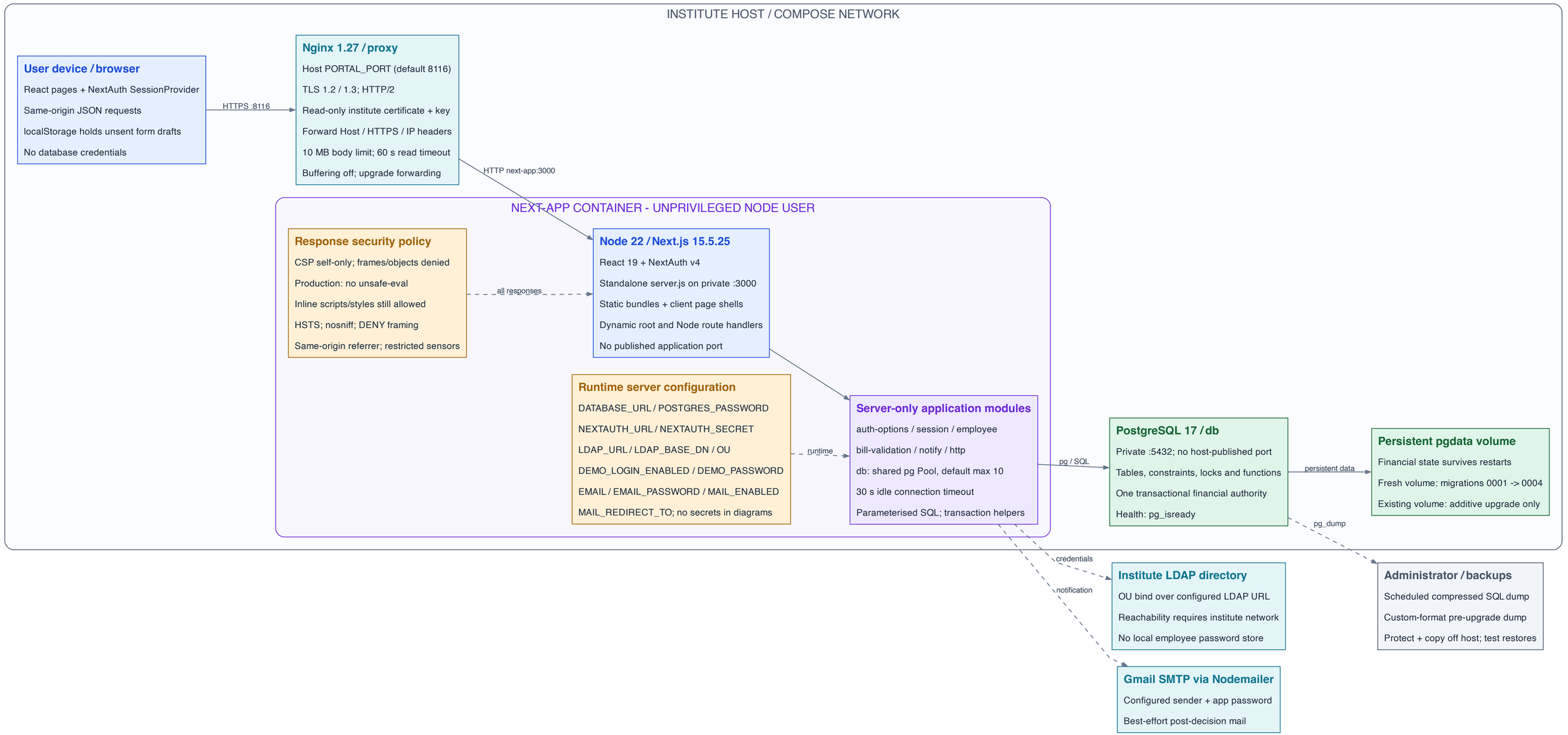
Identity -> role landing -> purchase workflow -> money, records and follow-up



- Store & Purchase is displayed as SNP in the reference; its stored role/status is Student Purchase.
- Hold is a desk sub-status, not a standalone bills.status. Correction resets the required route.
- All non-User roles can read all bills through the API; departmental lists and write scope are separate.
- QR links reopen an authenticated bill record, including records that are not yet accepted.

02 Runtime and trust boundaries

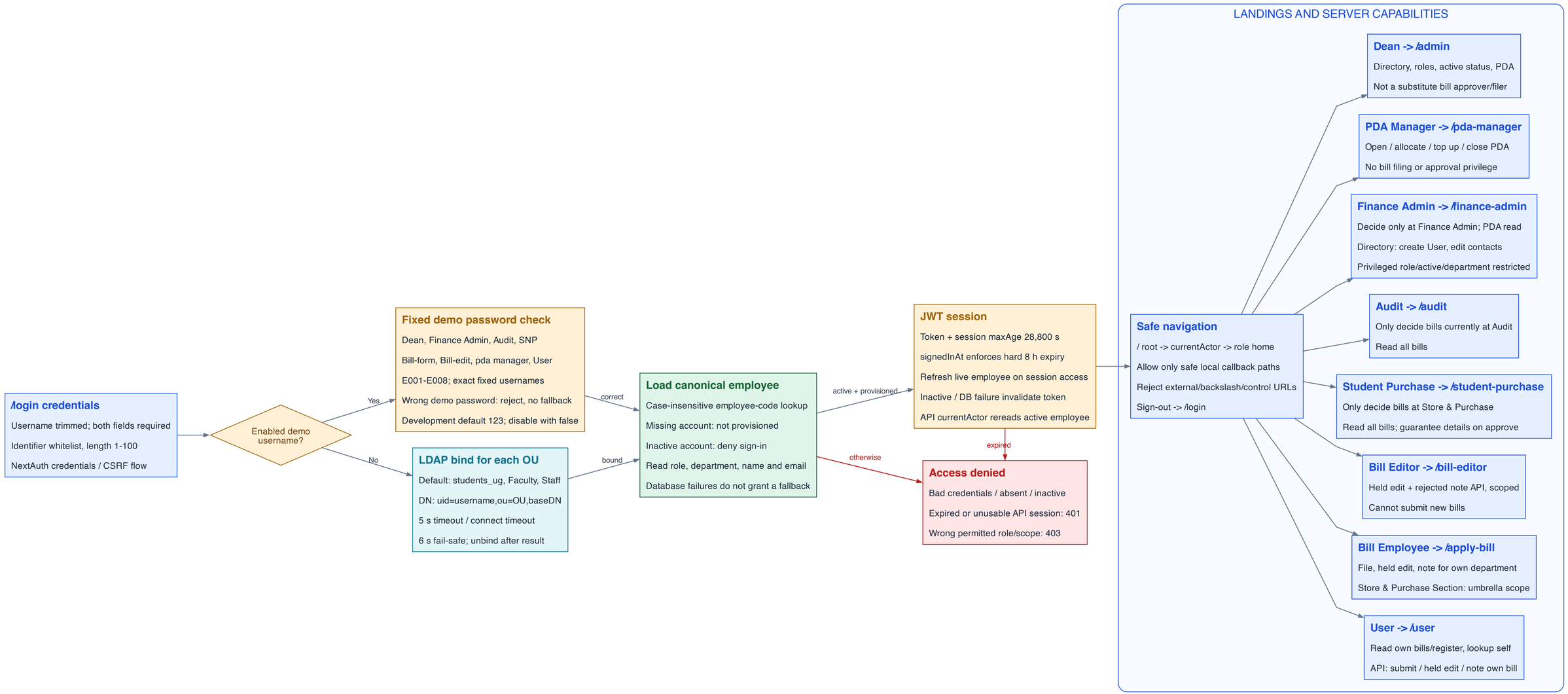
Self-hosted Docker: public HTTPS edge, private application and private database



- This is the configured runtime, not evidence that the current commit has been deployed.
- Only Nginx publishes a host port. PostgreSQL and Next.js stay on the private Compose network.
- There is no separate microservice, message broker, Redis cache, object store or Supabase client.
- Application health checks cover the login web tier; database readiness is a separate pg_isready check.

03 Authentication, session and role access

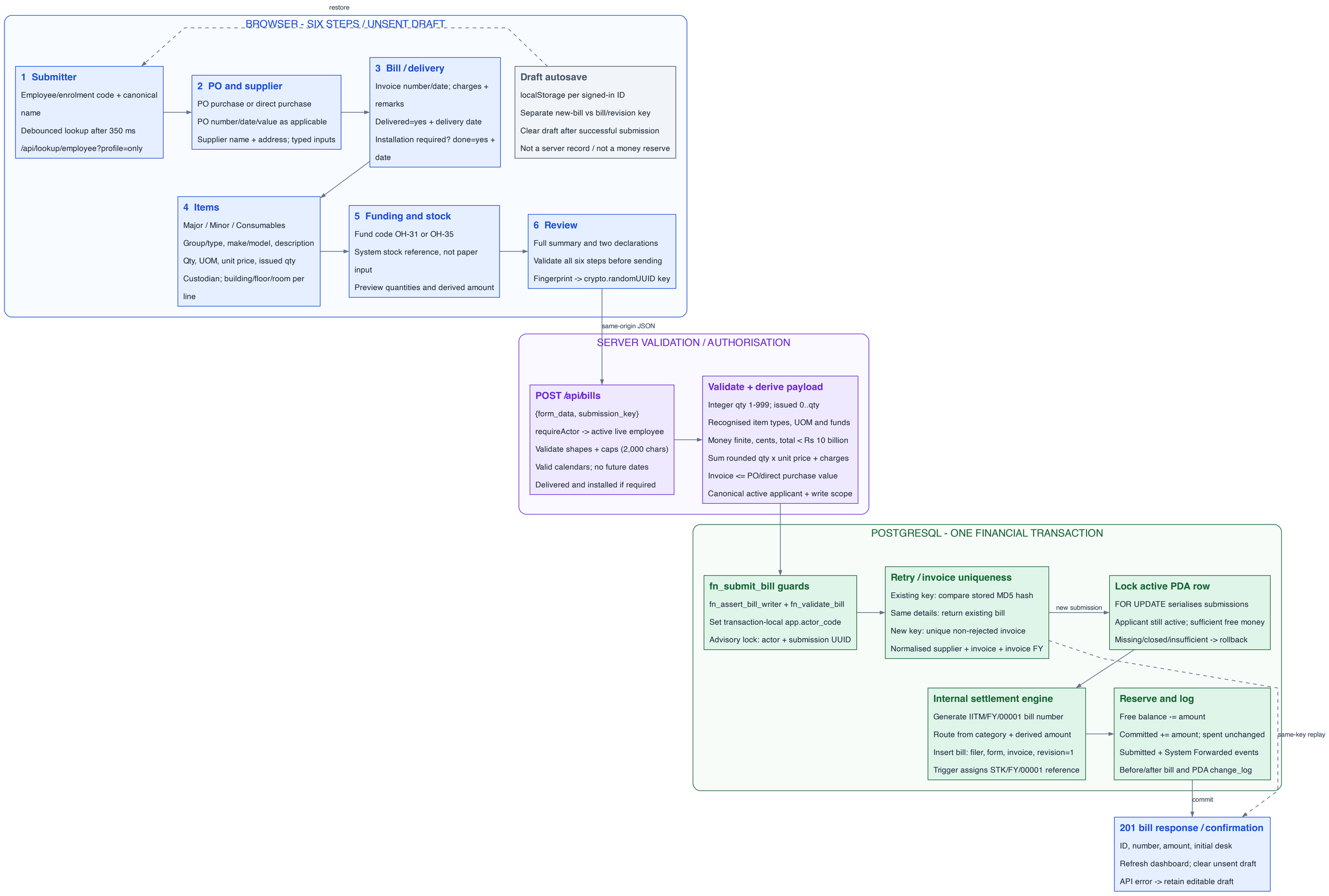
Authentication proves identity; employees defines permissions on every authenticated request



- Demo login remains intentionally available. Production needs explicit enablement and a non-empty password.
- Every signed-in role can read its own /api/me. All non-User roles have global bill/register read capability.
- The User UI currently displays bills/PDA; own submission/correction/acknowledgement are API capabilities, not a dedicated User wizard.
- Page redirects improve navigation; live API and SQL guards are the actual authorisation boundary.

04 Bill form and submission transaction

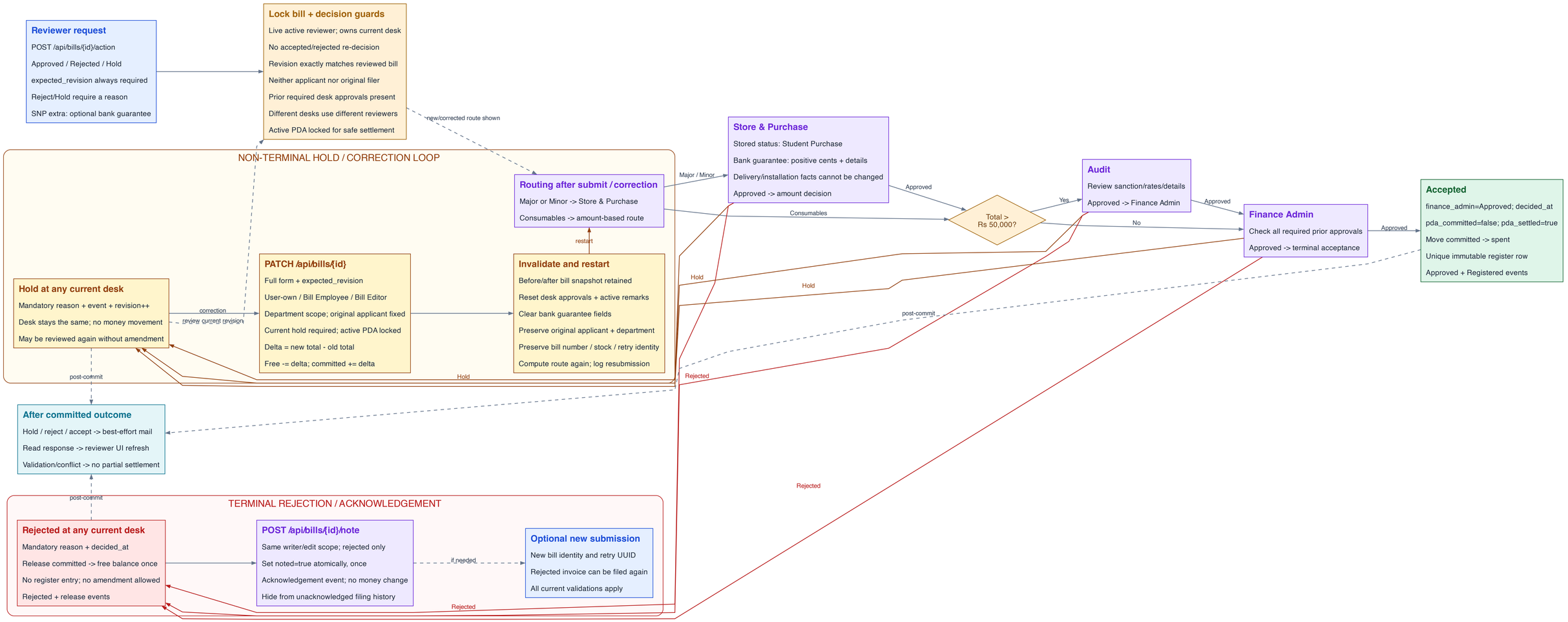
The full form survives end-to-end; the server computes money and PostgreSQL reserves it once



- A bill has 1-100 structured item lines, retained in bills.form_data; flat columns are compatibility/display fields.
- Client validation is guidance only. The API validates the form again and SQL independently checks money and workflow essentials.
- Same actor + retry key + same payload returns the existing bill; a changed payload using that key is rejected.
- Any failure rolls back the bill, reservation, counters and transactional audit/event writes together.

05 Approval, hold, correction and rejection

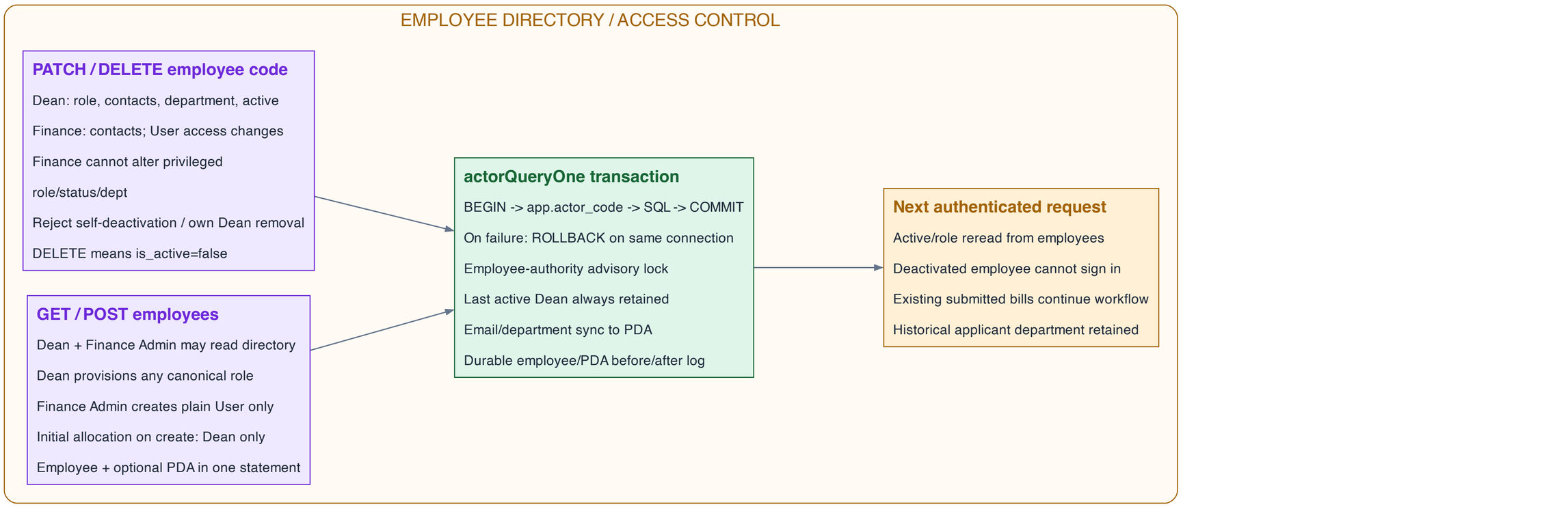
Database-authoritative state machine with revision checks and separation of duties



- Threshold is strictly greater than Rs 50,000. Exactly Rs 50,000 skips Audit.
- Hold leaves bills.status at its current desk and sets that desk's sub-status to Hold; funds remain reserved.
- A current desk may approve/reject an unchanged held bill; correction is not mandatory for every hold.
- The immutable event trail survives correction. Active approval/BG fields reset and displayed progress uses events since the last correction.

06 PDA money lifecycle and directory administration

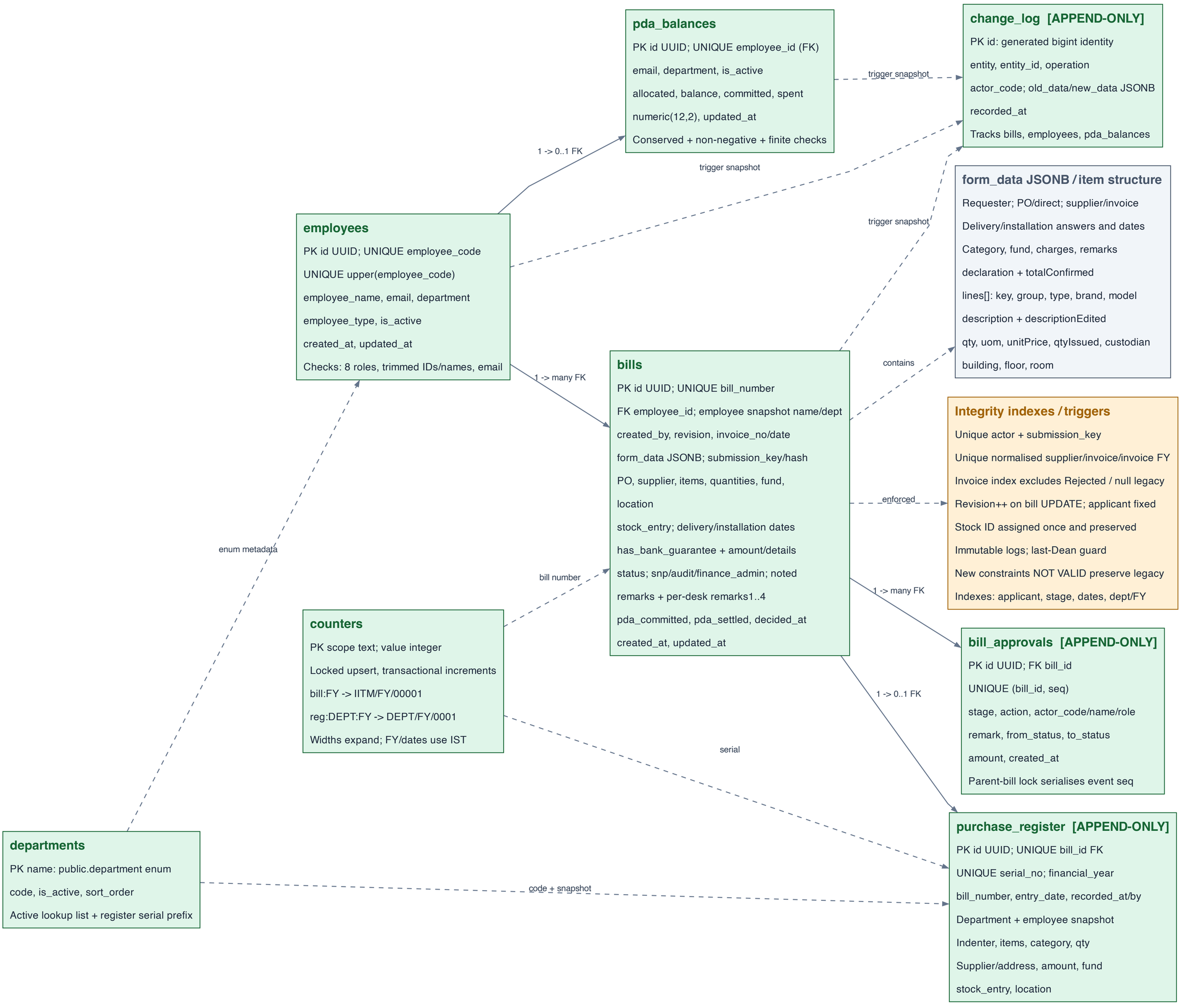
The conservation invariant is enforced in PostgreSQL, not by browser arithmetic



- allocated = balance + committed + spent; every component is non-negative and finite, stored as numeric(12,2).
- Hold keeps money committed; acknowledgement does not return money because rejection already did.
- Closing an account requires zero committed and zero spent and no open bills; free money can remain in the retained inactive row.
- SQL functions/locks enforce financial writes. Reads are scoped by the server; RLS is not enabled.

07 Database model, constraints and audit records

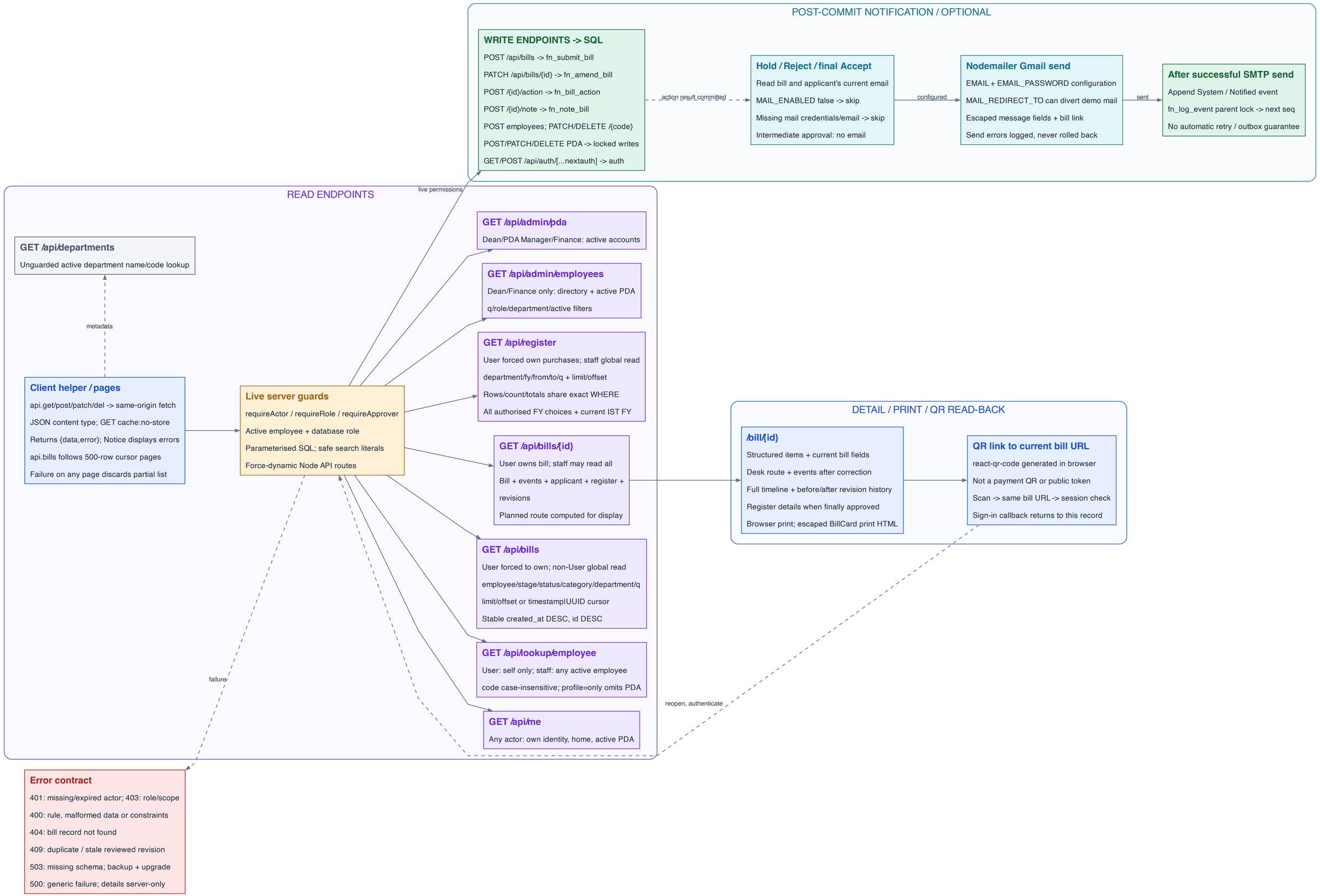
Eight public tables; structured item detail lives in JSONB, not a separate items table



- Solid relationships labelled FK are enforced foreign keys; dashed relationships are logical/snapshot links only.
- created_by, actor_code, recorded_by and change_log.entity_id are audit identities, not employee foreign keys.
- Employee department columns use public.department enum; departments supplies active labels/codes rather than an FK link.
- Append-only triggers block UPDATE/DELETE of ledgers; database owner/admin access must still be protected.

08 API surface, read flows, QR and notification

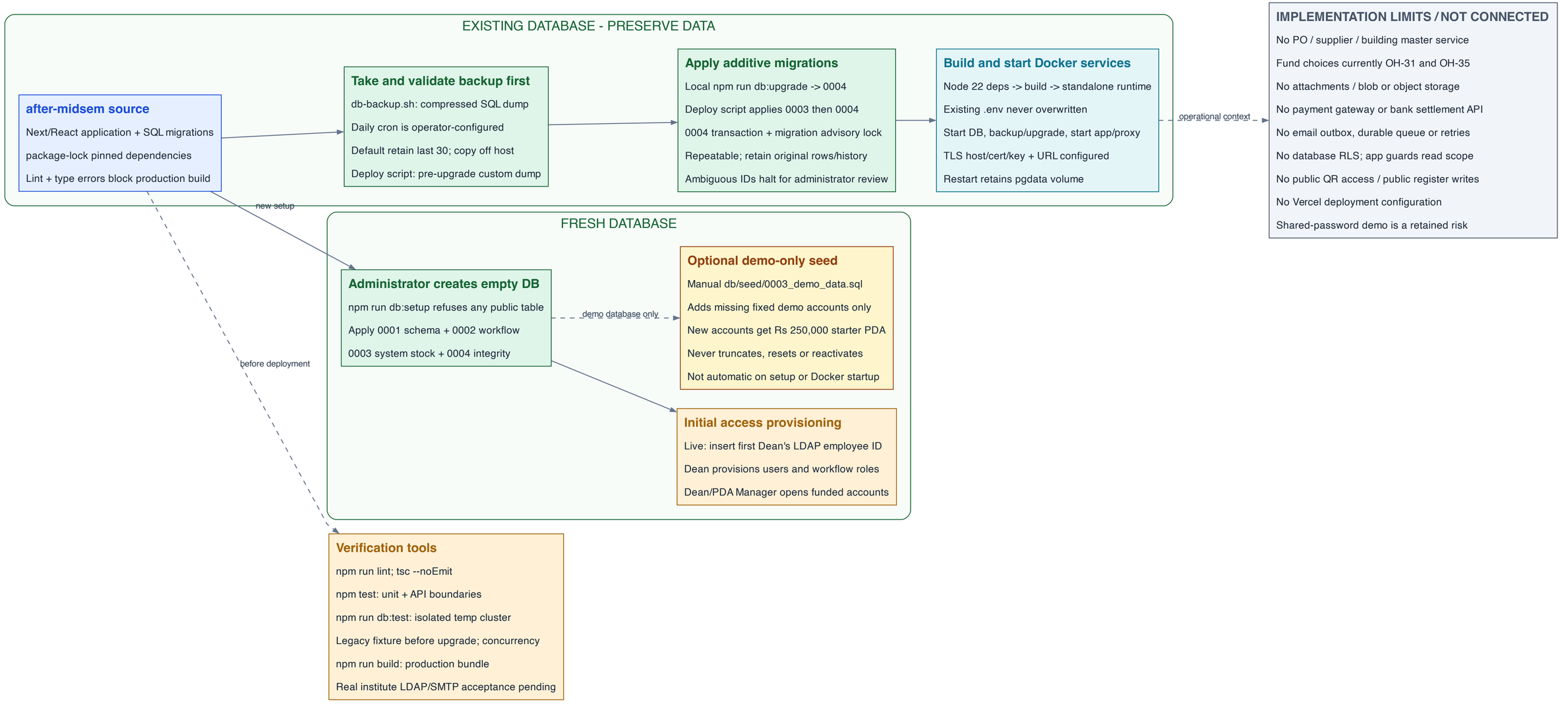
All financial reads/writes are mediated by Next.js route handlers; no browser-to-database access



- /api/departments has no session guard; it returns only active public department metadata.
- Staff global read access is current policy, not a promise of departmental confidentiality.
- Notification is awaited after the financial commit inside the request; it is not a durable queue or transactional outbox.
- The purchase register has no write endpoint. Only final Finance approval creates an entry.

09 Setup, deployment, verification and limits

Safe upgrade path preserves existing balances, bills, approvals and register history



- The diagram documents commit a96512e on after-midsem, including migration 0004; live schema/deployment was not inspected.
- 87 tests, ESLint, TypeScript and production build passed for the implementation before this diagram was authored.
- Migration 0004 preserves legacy rows; missing structured invoice/form fields need human completion on correction.
- Do not delete volumes or run base schema migration 0001 over an existing application database.

10 Source index and reading guide

01 Complete website flow

```
src/lib/roles.ts
src/app/page.tsx
src/server/auth-options.ts
src/app/api/bills/[id]/action/route.ts
db/migrations/0002_functions.sql
db/migrations/0004_integrity.sql
```

02 Runtime and trust boundaries

```
Dockerfile
docker-compose.yml
deploy/nginx/portal.conf
deploy/nginx/upgrade-map.conf
next.config.ts
src/server/db.ts
src/app/layout.tsx
```

03 Authentication, session and role access

```
src/server/auth-options.ts
src/server/session.ts
src/lib/roles.ts
src/lib/security.ts
src/app/login/page.tsx
src/app/page.tsx
src/app/api/admin/employees/[code]/route.ts
```

04 Bill form and submission transaction

```
src/app/apply-bill/UploadBill.tsx
src/app/apply-bill/bill-form/model.ts
src/app/apply-bill/bill-form/masters.ts
src/server/bill-validation.ts
src/app/api/bills/route.ts
db/migrations/0003_stock_entry.sql
db/migrations/0004_integrity.sql
```

05 Approval, hold, correction and rejection

```
src/lib/roles.ts
src/app/api/bills/[id]/action/route.ts
src/app/api/bills/[id]/route.ts
src/app/api/bills/[id]/note/route.ts
db/migrations/0002_functions.sql
db/migrations/0004_integrity.sql
src/app/student-purchase/page.tsx
```

06 PDA money lifecycle and directory administration

```
src/app/api/admin/pda/route.ts
src/app/api/admin/employees/route.ts
src/app/api/admin/employees/[code]/route.ts
src/server/db.ts
db/migrations/0001_schema.sql
db/migrations/0004_integrity.sql
```

07 Database model, constraints and audit records

```
db/migrations/0001_schema.sql
db/migrations/0002_functions.sql
db/migrations/0003_stock_entry.sql
db/migrations/0004_integrity.sql
src/types/database.ts
src/server/db.ts
```

08 API surface, read flows, QR and notification

```
src/app/api
src/lib/api.ts
src/server/http.ts
src/server/notify.ts
src/app/bill/[id]/page.tsx
src/app/register/page.tsx
src/components/BillCard.tsx
```

09 Setup, deployment, verification and limits

```
scripts/db-setup.sh
scripts/db-upgrade.sh
scripts/db-local.sh
scripts/deploy-server.sh
scripts/db-backup.sh
db/seed/0003_demo_data.sql
package.json
README.md
tests/db-integrity.test.ts
```

READING GUIDE

Solid arrow: primary flow / enforced FK when explicitly labelled. Dashed arrow: optional follow-up, configuration or logical link.

Blue: browser/page. Amber: identity/hold. Purple: API/workflow. Green: database/money. Red: failure/rejection. Cyan: external dependency.

Labels describe implementation at source commit a96512e. Diagram generation does not change application behavior or deploy migration 0004.

No real credentials, applicant records or production data are included. Original architecture.jpeg is preserved.